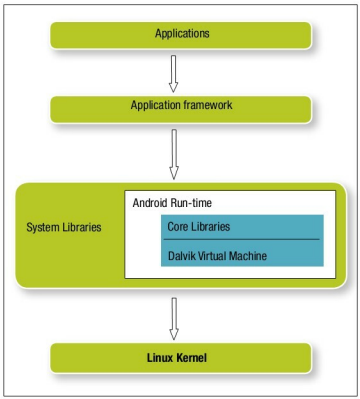

Figure 1: Android OS stack

Traditionally, when it came to choosing the VM architecture, people preferred stack-based machines, rather than the register-based architecture prevalent in most real processors. The main reason for this choice was the simplicity of the compiler, which emanates from the fact that stack instructions are small, and have implicit addressing. The basic criteria for choosing a particular VM implementation is the code size of the machine implementation (the smaller the better); and machine efficiency—which is measured in terms of the number of instructions needed to carry out an operation. Implementation-wise, instructions for a stack-based VM are shorter (since it'll have only POP/PUSH instructions) and are hence faster to fetch. A register-based VM has long instructions, and usually a quadruple of {*opcode, result, operand1, operand2*}. However, the advantage of the stack-based VM diminishes in terms of the number of instructions needed to perform an operation. A given task can often be expressed in fewer register-based VM instructions than what is required for a stack VM. For example, an addition operation "$a = b + c$" would need the following instructions for a stack VM:

```
I1: LOAD C
I2: LOAD B
I3: ADD
I4: STORE A
```

The equivalent operation in a register VM can be expressed in a single instruction:

```
I1 "add, a, b, c"
```

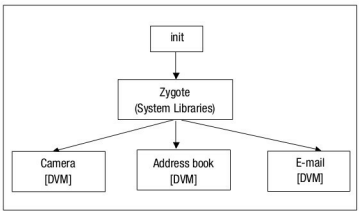

Figure 2: The Android runtime

The fewer the instructions, the smaller the number of instruction dispatches needed. (Instruction dispatch involves fetching/reading the instruction from memory, and jumping to the corresponding segment of VM code that implements the VM instruction.) Instruction dispatch is an expensive and often unpredictable operation, because in implementation it is essentially a 'switch-case', and hence a hard-to-predict indirect branch, unlike an 'if-else' condition.

There is a cost involved in the operand access too. In stack VMs, operands are accessed relative to the stack pointer, and hence explicit addressing for operands is not required. On the contrary, register VMs need to provide the location of all operands, by specifying the registers that carry those operands. Having all operands in the instruction has its benefits; the execution is faster compared to the stack VM, which needs a small calculation to find out the operand, while register VMs just read the registers.

Also, a stack VM might consume more memory cycles, since it uses a set of registers to simulate a stack. Besides, temporary values often spill out into the main memory, adding more memory cache cycles. In register VMs, temporary values usually remain in registers. Stack VMs are unable to use a few optimisations too. For example, in the case of common sub-expressions, which are recalculated each time they appear in the code, a register VM can calculate an expression once, and keep that in a register for all future references.

These reasons made a register VM model the preferred choice of implementation for the Dalvik VM, where quick execution is the foremost requirement.

The Android OS stack

Android is a complete software stack that comprises the Linux kernel, middleware and a few useful applications. It is developed, promoted, and maintained by Google. Its software stack is shown in Figure 1. The top layer comprises portable user applications, programmed in Java. These rely on the next layer, which exports a rich set of Java APIs, and provides a set of services and a system to facilitate applications, e.g., the content manager, resource manager, view services, etc. The next layer comprises C/C++ system